

Resumen Teórico: Autorreferencia, Recursión y Oráculos

Tópicos avanzados en Computabilidad — UNRC

Basado en el material de Pablo Castro (UNRC)

Índice

1. Introducción	1
2. Autorreferencia en máquinas de Turing	2
2.1. Idea general	2
2.2. El paso previo: la función q	2
2.3. La máquina <i>Self</i>	2
2.4. Analogía: párrafos que se reproducen	3
3. El Teorema de la Recursión	3
3.1. Enunciado y significado	3
3.2. Demostración (construcción de R)	4
4. Aplicación: A_{TM} es indecidible (otra vez)	5
5. Introspección en lenguajes de programación	5
5.1. Ejemplo en Python	5
6. El lenguaje de las MTS minimales	6
6.1. Definición	6
6.2. <i>Min</i> no es Turing reconocible	6
7. Oráculos y reducibilidad de Turing	7
7.1. Definición de oráculo	7
7.2. Resolver problemas indecidibles con oráculos	7
7.3. Reducibilidad de Turing	8
7.4. Comparación con la reducción de mapeo	8
8. Resumen final	8

1. Introducción

Hasta ahora vimos cómo las máquinas de Turing formalizan la noción de cómputo, cómo demostrar problemas indecidibles por diagonalización, y cómo extender esos resultados mediante reducciones. En este teórico estudiamos un grupo de *tópicos avanzados* que aprovechan una propiedad notable y a primera vista paradójica de las MTS: la **autorreferencia**.

Las máquinas de Turing pueden:

- Tomar como entrada *descripciones de otras MTs* (lo vimos en la máquina universal).
- Tomar como entrada *su propia* descripción.
- *Producir* descripciones de máquinas; en particular, su propia descripción.

Esto permite construir programas que *se ven a sí mismos*, demostrar el **Teorema de la Recursión** (una herramienta general para implementar autorreferencia), reobtener la indecidibilidad de A_{TM} de manera muy elegante, estudiar el lenguaje *Min* de las MTs minimales, y generalizar la noción de reducción mediante **oráculos**.

2. Autorreferencia en máquinas de Turing

2.1. Idea general

Una pregunta natural es: *¿puede una máquina de Turing escribir su propia descripción?* Si pensamos en programas, esto es lo que se llama un *quine*: un programa que, al ejecutarse, imprime su propio código fuente sin leer ningún archivo externo.

La respuesta es **sí**: existe una MT, que llamaremos *Self*, que con cualquier entrada termina escribiendo en la cinta su propia descripción $\langle Self \rangle$.

Intuición

La construcción es delicada porque “hablar de uno mismo” aparenta requerir conocer la propia descripción antes de existir. Pero usaremos una idea muy ingeniosa: dividir la máquina en dos partes A y B , donde A produce $\langle B \rangle$ y B usa $\langle B \rangle$ para *reconstruir* $\langle A \rangle$ y unirlos.

2.2. El paso previo: la función q

Proposición 2.1. Existe una función computable $q : \Sigma^* \rightarrow \Sigma^*$ tal que para toda cadena $w \in \Sigma^*$, $q(w)$ es la descripción $\langle P_w \rangle$ de una MT P_w que, con cualquier entrada, *imprime w y termina*.

Construcción de P_w . Dada w , la máquina P_w es muy simple: *ignora* su entrada, borra la cinta, escribe w y termina. Como diseñar esta máquina solo requiere “codificar” w en los estados/transiciones de P_w , esto se puede hacer algorítmicamente.

La MT M_q que computa q . Dada una entrada w , M_q genera (de manera mecánica) la descripción $\langle P_w \rangle$ de la MT P_w correspondiente.

Idea clave

q es una herramienta general: dado cualquier “valor” w , produce el código de un programa que imprime w . Es análogo a la operación `print(w)` considerada como “constructor de programas”.

2.3. La máquina *Self*

Usando q , construimos la máquina *Self* con dos partes consecutivas A y B :

Parte A. Es la máquina $P_{\langle B \rangle}$ obtenida aplicando q a $\langle B \rangle$, es decir, $A = P_{\langle B \rangle}$. Su efecto cuando se ejecuta es: *escribir $\langle B \rangle$ en la cinta y terminar*.

Parte B. Después de que se ejecutó A , la cinta contiene $\langle B \rangle$. Entonces B procede así:

1. Lee $\langle B \rangle$ de la cinta.

2. Aplica la función q a $\langle B \rangle$, obteniendo $\langle P_{\langle B \rangle} \rangle = \langle A \rangle$.
3. Escribe en la cinta la descripción completa $\langle AB \rangle = \langle Self \rangle$ (concatenando $\langle A \rangle$ adelante).

Resultado. Al terminar A y B , la cinta contiene $\langle AB \rangle = \langle Self \rangle$. La máquina ha escrito su propia descripción.

Intuición

¿Por qué funciona? Notar que A no necesita “saber su propia descripción”. A solo necesita conocer $\langle B \rangle$ (que sí podemos describir explícitamente: las instrucciones de B son fijas y conocidas). B , por su parte, reconstruye $\langle A \rangle$ de manera *computable* aplicando q a la información dejada por A . La autorreferencia se logra evitando la circularidad: A y B saben uno del otro, no de sí mismos.

2.4. Analogía: párrafos que se reproducen

Una analogía textual ilumina la idea de *Self*. Considera el siguiente par de instrucciones:

Copiar dos veces el siguiente párrafo, una con comillas dobles:
 “Copiar dos veces el siguiente párrafo, una con comillas dobles.”

Análisis.

- La primera línea (sin comillas) hace el rol de B : es la instrucción “activa” que dice qué hacer.
- La segunda línea (entre comillas) hace el rol de A : contiene la *representación* de B como dato.

Al “ejecutar” las instrucciones uno escribe primero la versión *con* comillas (es decir, la representación de B) y luego la versión sin comillas (es decir, B mismo). El resultado es una copia completa del par original.

Idea clave

La autorreferencia es posible porque hay dos representaciones de la “misma información”: una *ejecutable* (el código) y otra *como dato* (la cadena entre comillas). La parte ejecutable puede manipular la versión-dato y reconstruir todo.

3. El Teorema de la Recursión

La idea de la sección anterior se puede generalizar enormemente: en cualquier modelo de cómputo razonable, podemos *escribir programas que tienen acceso a su propio código*. La formalización de este hecho es el *Teorema de la Recursión*.

3.1. Enunciado y significado

Teorema 3.1 (Teorema de la Recursión). Sea $t : \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$ una función computable, computada por una MT T . Entonces existe una función computable $r : \Sigma^* \rightarrow \Sigma^*$, computada por una MT R , tal que para toda w ,

$$r(w) = t(\langle R \rangle, w).$$

Intuición

T es una máquina que “ejecuta otra máquina sobre una entrada”: dada $\langle M \rangle$ y w devuelve el resultado de correr M sobre w . El teorema garantiza que podemos construir una MT R que, sobre w , computa $t(\langle R \rangle, w)$. Es decir, R puede usar su propio código $\langle R \rangle$ como parte de su cómputo.

3.2. Demostración (construcción de R)

La construcción es análoga a la de *Self*, con un pequeño agregado: R se compone de tres partes A , B y T .

Dada la MT T que computa t , definimos:

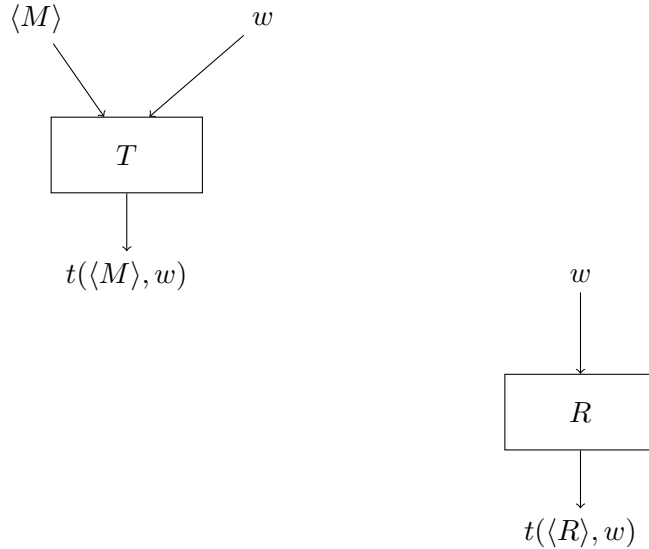
Parte A. Es la máquina $P_{\langle BT \rangle}$, es decir, una MT que con cualquier entrada w , deja en la cinta la secuencia $w\langle BT \rangle$. Es decir, A preserva la entrada w y agrega después $\langle BT \rangle$.

Parte B. Hace lo siguiente:

1. Lee $\langle BT \rangle$ de la cinta.
2. Aplica q a $\langle BT \rangle$, obteniendo $\langle P_{\langle BT \rangle} \rangle = \langle A \rangle$.
3. Concatena para formar $\langle ABT \rangle = \langle R \rangle$.
4. Escribe en la cinta $\langle R \rangle w$ (la descripción de la máquina completa seguida de la entrada original).

Parte T. Se ejecuta sobre el contenido actual de la cinta, que es $\langle R \rangle w$. Por definición, T con esa entrada calcula $t(\langle R \rangle, w)$.

Resultado. Sobre la entrada w , la MT $R = ABT$ termina con $t(\langle R \rangle, w)$ en la cinta. Esta es precisamente la función r buscada. $\square$



Intuición

El teorema dice: si tenemos una operación t que “ejecuta otra máquina”, podemos *plegar* esa operación para que la máquina ejecute *a sí misma*. Como consecuencia, al programar podemos suponer que el código tiene acceso a su propia descripción $\langle R \rangle$ “gratis”.

4. Aplicación: A_{TM} es indecidible (otra vez)

El Teorema de la Recursión brinda una prueba alternativa, muy corta y elegante, de la indecidibilidad del Halting Problem.

Teorema 4.1. A_{TM} es indecidible.

Prueba (con Teorema de la Recursión). Supongamos por absurdo que existe una MT H que decide A_{TM} , es decir,

$$H(\langle M, w \rangle) = \begin{cases} \text{acepta} & \text{si } M \text{ acepta } w, \\ \text{rechaza} & \text{en otro caso.} \end{cases}$$

Usamos el Teorema de la Recursión para construir una MT B con acceso a su propia descripción $\langle B \rangle$, que sobre la entrada w hace lo siguiente:

1. Obtiene su propia descripción $\langle B \rangle$ (lo permite el Teorema de la Recursión).
2. Ejecuta H con la entrada $\langle B, w \rangle$.
3. Si H acepta, B **rechaza**.
4. Si H rechaza, B **acepta**.

Contradicción. Por construcción, B hace *exactamente lo contrario* de lo que predice H sobre $\langle B, w \rangle$:

- Si B acepta w , entonces H había rechazado $\langle B, w \rangle$, lo que significa que B no acepta w . Contradicción.
- Si B rechaza w , entonces H había aceptado $\langle B, w \rangle$, lo que significa que B acepta w . Contradicción.

Por lo tanto, H no puede existir. □

Idea clave

Comparada con la prueba original por diagonalización, esta versión es mucho más directa: no hace falta construir ninguna “máquina diagonal” externa ni razonar sobre tablas. El Teorema de la Recursión *empaqueta* la diagonalización en una herramienta de propósito general, y el argumento se vuelve casi una línea: “si H existiera, construimos B que hace lo contrario de lo que H predice sobre B mismo”.

5. Introspección en lenguajes de programación

La autorreferencia no es solamente una curiosidad teórica: los lenguajes de programación reales implementan formas de *introspección* (también llamada *reflexión*) que permiten que un programa lea su propio código en tiempo de ejecución.

5.1. Ejemplo en Python

En Python, la biblioteca `inspect` permite que un programa obtenga información sobre sus propias funciones, clases, atributos y código fuente:

```
import inspect

def foo():
```

```

    pass

print(inspect.getmembers(foo)) # info sobre la función

import inspect

def foo(x, y, z):
    pass

print(inspect.getargspec(foo)) # info sobre argumentos

```

Intuición

Por el Teorema de la Recursión, cualquier lenguaje Turing-completo *podría* ofrecer introspección. Los lenguajes la implementan de manera nativa porque resulta práctica: depuración, serialización, frameworks de testing, generación de código en tiempo de ejecución, etc. La autorreferencia teórica de las MTs aparece, transformada, en cada programa que se “mira a sí mismo”.

6. El lenguaje de las MTs minimales

Una aplicación más sutil de la autorreferencia tiene que ver con la noción de *minimalidad*.

6.1. Definición

Definición 6.1 (MT minimal). Una MT T se dice **minimal** si no existe otra MT equivalente a T (es decir, que reconozca el mismo lenguaje) con descripción *más corta*, $|\langle T' \rangle| < |\langle T \rangle|$.

Definimos el lenguaje

$$Min = \{\langle M \rangle \mid M \text{ es una MT minimal}\}.$$

6.2. Min no es Turing reconocible

Teorema 6.2. Min no es Turing reconocible.

Prueba (con Teorema de la Recursión). Supongamos, por absurdo, que Min es Turing reconocible. Entonces existe un enumerador E que lista todas las MTs minimales (recordemos que un lenguaje es reconocible sii hay un enumerador que lo enumera).

Usando el Teorema de la Recursión, construimos una MT M con acceso a su propia descripción $\langle M \rangle$, que sobre cualquier entrada hace lo siguiente:

1. Ejecuta E y va recibiendo, una a una, las descripciones de MTs minimales que E va imprimiendo.
2. En cuanto aparece una MT D tal que $|\langle D \rangle| > |\langle M \rangle|$ (es decir, D es estrictamente más larga que M):
3. M simula D con la entrada original, y devuelve lo que D devuelva.

Análisis.

- Como E enumera *todas* las MTs minimales, y hay infinitas (las hay arbitrariamente grandes), eventualmente aparecerá alguna D con $|\langle D \rangle| > |\langle M \rangle|$.

- Por construcción, M se comporta como D en cualquier entrada: $\mathcal{L}(M) = \mathcal{L}(D)$.
- Pero $\langle M \rangle$ es *más corto* que $\langle D \rangle$. Entonces D no podía ser minimal: contradicción.

Por lo tanto, Min no puede ser reconocible. $\square$

Intuición

La idea es bellísima: una MT “espera” hasta ver una hermana más larga que ella, y entonces se hace pasar por esa hermana. Como la hermana se suponía minimal (no debería tener equivalente más corto), aparece la contradicción. El Teorema de la Recursión es esencial: sin él, M no podría conocer su propia longitud $|\langle M \rangle|$ para hacer la comparación.

7. Oráculos y reducibilidad de Turing

Hasta ahora estudiamos máquinas que computan “por sí solas”. Una generalización muy poderosa es permitirles *consultar a un oráculo*: un dispositivo que responde correctamente preguntas de pertenencia a un lenguaje fijo, posiblemente indecidible.

7.1. Definición de oráculo

Definición 7.1 (Oráculo). Un **oráculo** para un lenguaje B es un dispositivo externo que, dada una cadena w , responde correctamente y en un solo paso si $w \in B$ o $w \notin B$.

Definición 7.2 (MT con oráculo). Una **máquina de Turing con oráculo** es una MT extendida con la capacidad de consultar a un oráculo. Si la máquina M usa el oráculo B , escribimos M^B .

Intuición

Pensemos al oráculo como una caja negra mágica: si tuviéramos uno para A_{TM} , podríamos preguntarle “¿ M acepta w ?” y recibir la respuesta correcta instantáneamente. Esto es claramente *no realista*, pero es una herramienta teórica muy útil para clasificar la dificultad relativa de los problemas.

7.2. Resolver problemas indecidibles con oráculos

Si tenemos un oráculo para un problema suficientemente difícil, podemos resolver otros problemas que serían indecidibles por sí solos.

Ejemplo 7.3. Con un oráculo para A_{TM} , podemos decidir E_{TM} (el vacío del lenguaje).

Construcción. Dada $\langle M \rangle$ queremos decidir si $\mathcal{L}(M) = \emptyset$. Construimos la MT N que con cualquier entrada hace lo siguiente:

1. Para $k = 1, 2, 3, \dots$:
 - Por cada cadena s de longitud k : simular M con la entrada s por k pasos.
 - Si *alguna* de esas simulaciones acepta, N acepta.

Observación clave: N acepta *cualquier* entrada ε si y solo si M acepta *alguna* cadena, es decir, $\mathcal{L}(M) \neq \emptyset$.

Ahora consultamos al oráculo para A_{TM} con la pregunta “¿ N acepta ε ?”. La respuesta nos dice si $\mathcal{L}(M)$ es vacío o no.

7.3. Reducibilidad de Turing

La noción de oráculo permite definir un tipo de reducción más general que la que vimos antes.

Definición 7.4 (Decidibilidad relativa). Un lenguaje A es **decidible relativo a B** si existe una MT M^B (con acceso al oráculo B) que decide A .

Definición 7.5 (Reducción de Turing). Decimos que A es **Turing reducible** a B , escrito $A \leq_T B$, si A es decidible relativo a B .

Ejemplo 7.6. Por la construcción anterior, $E_{TM} \leq_T A_{TM}$: con un oráculo para A_{TM} , podemos decidir E_{TM} .

7.4. Comparación con la reducción de mapeo

La reducción de Turing $\leq_T$ es *más general* que la reducción de mapeo $\leq$ vista en el teórico de Reducibilidad:

- Si $A \leq B$ (reducción de mapeo), entonces $A \leq_T B$. La reducción de mapeo es un caso particular: traducir la entrada y hacer una sola consulta.
- La recíproca no vale: hay lenguajes con $A \leq_T B$ pero no $A \leq B$. Por ejemplo, $\overline{A_{TM}} \leq_T A_{TM}$ (con un oráculo para A_{TM} , simplemente invierto la respuesta), pero $\overline{A_{TM}} \not\leq A_{TM}$ (si valiera la reducción de mapeo, como A_{TM} es reconocible, también $\overline{A_{TM}}$ lo sería, lo cual contradice resultados conocidos).

Idea clave

La reducción de Turing es más expresiva porque permite *múltiples* consultas al oráculo, en cualquier orden, y combinar las respuestas como uno quiera. La reducción de mapeo, en cambio, exige una única transformación de la entrada y una única consulta al decididor de B . La diferencia se vuelve crucial cuando se estudian las relaciones de reducibilidad entre lenguajes reconocibles y no reconocibles.

8. Resumen final

Las ideas centrales de este teórico:

1. **Autorreferencia.** Las MTs pueden tomar como entrada descripciones de MTs (incluso la propia) y también producirlas. Esto da origen a un fenómeno análogo a los *quines* de la programación.
2. **La máquina *Self*.** Mediante una función computable q (que dada w produce $\langle P_w \rangle$, una MT que imprime w), construimos $Self = AB$ con $A = P_{\langle B \rangle}$ y B usa $q(\langle B \rangle)$ para reconstruir $\langle A \rangle$ y armar $\langle Self \rangle$. La clave es *evitar la circularidad*: cada parte conoce a la otra, no a sí misma.
3. **Analogía textual.** El truco del párrafo “Copiar dos veces. . . ” (una vez sin comillas y otra entre comillas) es la versión informal del mismo principio: separar el “código” del “dato”.
4. **Teorema de la Recursión.** Si t es computable, existe R tal que $r(w) = t(\langle R \rangle, w)$. En lenguaje informal: *cualquier MT puede usar su propia descripción*. Garantiza que la autorreferencia es siempre legítima.
5. **Nueva prueba de la indecidibilidad de A_{TM} .** Con el Teorema de la Recursión, construimos una MT B que hace lo contrario de lo que predice el supuesto decididor H sobre

$\langle B, w \rangle$, y obtenemos una contradicción inmediata. Es la diagonalización “compilada” como teorema.

6. **Introspección.** En los lenguajes de programación reales, la autorreferencia se manifiesta como *reflexión*: bibliotecas como **inspect** en Python permiten que un programa lea su propio código.
7. **Min no es reconocible.** Con el Teorema de la Recursión, una MT puede esperar hasta ver una versión más larga de sí misma en la enumeración y hacerse pasar por esa máquina; contradice la minimalidad.
8. **Oráculos.** Una MT con oráculo B , escrita M^B , puede consultar gratis la pertenencia a B . Esto permite resolver problemas indecidibles cuando el oráculo es suficientemente potente.
9. **Reducción de Turing** $A \leq_T B$: A es decidable relativo a B . Es estrictamente más expresiva que la reducción de mapeo (permite múltiples consultas al oráculo y combinarlas libremente).

Intuición

La autorreferencia es la herramienta central en muchos resultados profundos de la lógica y la computabilidad: la prueba de Turing del Halting Problem, el primer Teorema de Incompletitud de Gödel, las paradojas semánticas (mentiroso, Russell), los virus informáticos, y los lenguajes de programación con reflexión. Todos comparten la misma estructura: *construir un objeto que habla de sí mismo* y derivar de allí, o bien resultados imposibles, o bien herramientas extraordinariamente flexibles.